

sUNIT -5

Templates and Exception handling:

Introduction, Class Templates, Class Templates with Multiple Parameters, Function Templates, Function Templates with Multiple Parameters, Member Function Templates.

Exception Handling :Basics of Exception Handling, Types of exceptions, Exception Handling Mechanism, Throwing and Catching Mechanism, Rethrowing an Exception, Specifying Exceptions.

GENERIC PROGRAMMING(**Templates**)

Generic programming is an approach where generic types are used as parameters in algorithms so that they work for a variety of suitable data types and data structures.

A significant benefit of object oriented programming is reusability of code which eliminates redundant coding. An important feature of C++ called templates strengthens this benefit of OOP and provides great flexibility to the language. Templates support generic programming, which allows to develop reusable software components such as functions, classes etc.. supporting different data types in a single framework.

Templates Concept

Introduction

Instead of writing different functions for the different data types, we can define common function.

For example

```
int max(int a,int b); // Returns maximum of two integers float
max(float a,float b); // Return maximum of two floats char max(char
a,char b); // Returns maximum of two characters
(this is called as function overloading)
```

But, instead of writing three different functions as above, C++ provided the facility called "Templates". With the help of templates you can define only one common function as follows:

```
T max(T a,T b); // T is called generic data type
```

Template functions are the way of making function/class abstracts by creating the behavior of function without knowing what data will be handled by a function. In a sense this is what is known as “generic functions or programming”.

Template function is more focused on the algorithmic thought rather than a specific means of single data type. For example you could make a templated stack push function. This push function can handle the insertion operation to a stack on any data type rather than having to create a stack push function for each different type.

Syntax:

```
template < class type > ret_type
fun_name(parameter list)
{
    -----//body of function
    -----
} //www.suhritsolutions.com
```

Features of templates:-

1. It eliminates redundant code
2. It enhances the reusability of the code.
3. It provides great flexibility to language

Templates are classified into two types. They are

1 .Function templates

2.Class Templates.

Function Templates

The templates declared for functions are called as function templates. A function template defines how an individual function can be constructed. Syntax :

```
template < class type,.....>
ret _type fun _ name(arguments)
{
    -----// body of the function
    -----
}
```

CLASS TEMPLATES

The templates declared for classes are called class templates. A class template specifies how individual classes can be constructed similar to the normal class specification. These classes model a generic class which support similar operations for different data types. General Form of a Class Template

```
template <class T>
class class-
name {
    .....
    .....
};
```

A class created from a class template is called a template class. The syntax for defining an object of a template class is:

```
classname<type> objectname(arglist);
```

```
#include<iostream.h>
#include<conio.h>
template <class T> class
swap
{ T
a,b;
public:
swap(T x,T y)
{ a=x;
b=y;
}
void swapab()
```

```

{
T temp;
temp=a;
    a=b;
b=temp;
}
void showdata()
{ cout<<a<<b;
}
};

void main()
{
int m,n;
float m1,n1;
cout<<"Enter integer values"; cin>>m>>n;
cout<<"Enter floating
values"; cin>>m1>>n1;
swap<int> c1(m,n);
swap<float> c2(m1,n1);
c1.swapab(); c1.showdata();
c2.swapab(); c2.showdata();
}

```

Class Template with Multiple Parameters

Syntax:

```

template <class T1, class T2,...>
    class class-name {
.....
.....
}

```

```

#include<iostream.h>
    template <class
T1,class T2> class Test {
T1 a; T2
b;
public:

```

```

Test(T1 x,T2 y)
{ a=x; b=y;
} void
show() {
cout<<a<<b;
}
};
void main()
{

```

```

Test<float,int> test1(1.23,123);
Test<int,char> test2(100,"w");
test1.show(); test2.show();
}

```

FUNCTION TEMPLATES

Like class template we can also define function templates that would be used to create a family of functions with different argument types.

General Form:

```

template <class T>
return-type function-name (arguments of type T) {
.....
.....
}
#include<iostream.h>
template<class T>
void swap(T &x, T &y)
{
T temp = x; x=y;
y=temp; }
void fun(int m,int n,float a,float b)
{
cout<<m<<n;
swap(m,n);
cout<<m<<n;
cout<<a<<b; swap(a,b);
cout<<a<<b;
} int
main() {
fun(100,200,11.22,33.44); return
0;
}

```

Example 2:-

```

#include < iostream.h >
#include < conio.h >
template T max(T a, T
b)
{
if(a>b)
return a;
else
return b;
} void
main( )
{ char ch1,ch2,ch3;
cout<<"enter two characters"<< ch2<<
ch3; cin>>ch2>>ch3; d=max(ch2,ch3);
cout<<"max(ch2,ch3)"<< ch1;
int a,b,c; cout<<"enter two
integers:"; cin>>a>>b;

```

```

    c=max(a,b);
    cout<<"max(a,b):"<< c<< endl;
    float f1,f2,f3;
    cout<<"enter two floats< f1 f2
    >:"; cin>>f1,f2; f3=max(f1,f2);
    cout<<"max(f1,f2):"<< f3;
}

```

output:

```

enter two characters: A,B
max(ch2,ch3):B enter two
integers:20,10 max (a,b)
:20 enter two floats
:20.5,30.9 max (f1,f2)
:30.9

```

Function Template with Multiple Parameters

Like template class, we can use more than one generic data type in the template statement, using a comma -separated list as shown below: template <class T1, class T2,> return-type function- name(arguments of types T1,T2.) {

```

.....
.....
}

```

```

#include<iostream.h>
#include<string.h>
template<clas T1, class T2> void
display(T1 x,T2 y)
{
    cout<<x<<y;
} int
main()
{
    display(1999,"EBG");

    display(12.34,1234); return
    0;
}

```

Exception handling

Exceptions: Exceptions are runtime anomalies or unusual conditions that a program may encounter while executing .Anomalies might include conditions such ass division by zero, accessing an array outside of its bounds or running out of memory or disk space. When a program encounters an exception condition, it must be identified and handled.

Exceptions provide a way to transfer control from one part of a program to another. C++ exception handling is built upon three keywords: try, catch, and throw.

Types of exceptions:There are two kinds of exceptions

- 1.Synchronous exceptions

2. Asynchronous exceptions

1. Synchronous exceptions: Errors such as “Out-of-range index” and “over flow” are synchronous exceptions

2. Asynchronous exceptions: The errors that are generated by any event beyond the control of the program are called asynchronous exceptions

The purpose of exception handling is to provide a means to detect and report an exceptional circumstance

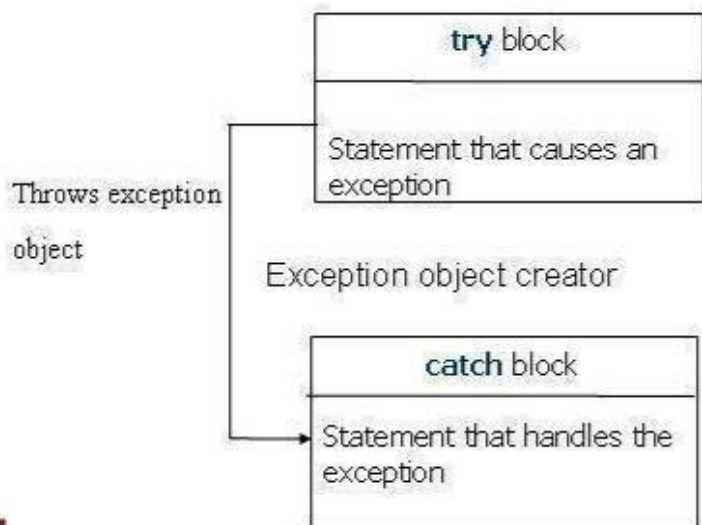
Exception Handling Mechanism:

An exception is said to be thrown at the place where some error or abnormal condition is detected. The throwing will cause the normal program flow to be aborted, in a raised exception. An exception is thrown programmatic, the programmer specifies the conditions of a throw.

In handled exceptions, execution of the program will resume at a designated block of code, called a catch block, which encloses the point of throwing in terms of program execution. The catch block can be, and usually is, located in a different function than the point of throwing.

C++ exception handling is built upon three keywords: try, catch, and throw.

Try is used to preface a block of statements which may generate exceptions. This block of statements is known as try block. When an exception is detected it is thrown by using throw statement in the try block. Catch block catches the exception thrown by throw statement in the try block and handles it appropriately.



```
#include<iostream>
```

```
try  
{
```

```
-----  
-----  
throw val;  
-----  
-----
```

throws
exception
value

```
}  
catch(data-type arg)  
{
```

```
-----  
-----  
-----
```

```
}
```

```
using
```

```
namespace std; int
```

```
main() {
```

```
int a,b;
```

```
cout<<"Enter any two integer values";
```

```
cin>>a>>b;
```

```
int x=a-b; try
```

```
{ if(x!=0)
```

```
{ cout<<"Result(a/x)="<<a/x<<endl;
```

```
} else {
```

```
throw x;
```

```
}
```

```
} catch(int ex)
```

```
{
```

```
cout<<"Exception caught:Divide By Zero \n"; }
```

```
}
```

THROWING MECHANISM

When an exception is detected, it can be thrown by using throw statement in any one of the following forms

- throw(exception);
- throw exception;
- throw; CATCHING MECHANISM:

Catch block is as below

```
Catch(data type arg)
```

```
{
```

```
//statements for handling
```

```
//exceptions
```

```
}
```

Multiple catch statements: try

```
{
//try block }
catch(data type1 arg)
{
//catch block1
}
catch(data type2 arg)
{
//catch block2 }
.....
.....
catch(data typeN arg)
{
//catch blockN
}
```

- When an exception is thrown, the exception handler are searched in order fore an appropriate match.
- It is possible that arguments of several catch statements match the type of an exception. In such cases the first handler that matches the exception type is executed

Write a Program to catch multiple catch statements

```
#include<iostream.h>
void test(int x)
{
    try
    { if(x==1) throw x; else
      if(x==0) throw 'x'; else
      if(x== -1) throw 1.0;
      cout<<"end of try block"<<endl;
    }
    catch(char c)
    {
    cout<<"caught a character"<<endl;
    }
    catch(int m)
    {

    cout<<"caught an integer"<<endl;
    }
    catch(double d)
    {
    cout<<"caught a double"<<endl;
    }
}
int
main() {
test(1);
test(0);
```



```
test(-1);
test(2);
return 0;
}
```

Output:

```
caught an integer
caught a character
caught a double
end of try block
```

Catch All Exceptions:

all possible types of exceptions and therefore may not be able to design independent **catch** handlers to catch them. In such circumstances, we can force a **catch** statement to catch all exceptions instead of a certain type alone.

```
catch(...)
{
.....
}
```

Write a Program to catch all exceptions

```
#include<iostream.h>
void test(int x)
{
try
{
if(x==0) throw x;
if(x==0) throw 'x'; if(x==1) throw 1.0;
} catch(...)
{
cout<<"caught exception"<<endl;
} } int
main() {
test(-1);
test(0);

test(1);

return 0;
}
```

Re-throwing an Exception:

It is possible to pass exception caught by a catch block again to another exception handler. This is known as Re-throwing.

```

#include <iostream>
using namespace std;
void MyHandler()
{ try
  { throw "hello";
  }
  catch (const char*)
  {
    cout <<"Caught exception inside MyHandler\n";
    throw; //rethrow char* out of function }
} int
main()
{ cout<< "Main start...."<<endl;
  try
  {
    MyHandler();
  }
  catch(const char*)
  { cout <<"Caught exception inside Main\n";
  } cout << "Main end";
  return 0;
}

```

Specifying Exceptions:

Specification of exception restrict functions to throw some specified exceptions only with the use of throw(exception list) in the the header of the function.

General form

Type function_name(argument list) throw(exception -list)

```

{
Statements try
{ statements
}

```

```

}

```

```

#include <iostream>
using namespace std; void
test(int x)
throw(int,float,char)
{ switch(x) { case
  1:throw x; break;
  case 2:throw 'x'; break;
  case 3:throw double(x); break;
  case 4:throw float(x); break;
  }
} int
main()
{ try
  { test(4); //test(4) leads to abnormal termination
  }
}

```

```
    catch(int i)
    { cout <<"Caught int type exception\n";
    } catch(float
f)
    { cout <<"Caught float type exception\n";
    } catch(char
c)
    { cout <<"Caught char type exception\n";
    } catch(double
i)
    { cout <<"Caught Double type exception\n";
    }

    return 0;
}
```